



Learning from Unlabeled Data for $A\beta$ Plaque Type Classification

A Comparative Study of Supervised, Semi-Supervised, and Self-Supervised Learning
on Segmented Brain Pathology Crops

< Parham Bateni¹ >
Supervisor(s): <Prof. Marcel Reinders¹>, <Xucong Zhang¹>
¹EEMCS, Delft University of Technology, The Netherlands

Honours Project Report Submitted to EEMCS Faculty Delft University of Technology,
In Partial Fulfilment of the Requirements
For the Bachelor of Computer Science and Engineering
May 12, 2026

Abstract

Expert annotations in histopathology are expensive and limited, while large collections of unlabeled image crops are often available through automated preprocessing pipelines. This work investigates whether unlabeled amyloid- β ($A\beta$) plaque images can improve multi-class plaque-type classification when combined with a small expert-annotated dataset. Using a shared ResNet-18 backbone, we compare supervised learning with three semi-supervised methods (Pi-Model, FixMatch, and Mean Teacher) and two self-supervised approaches (SimCLR and a variational autoencoder) on the same labeled and unlabeled plaque pool. All methods are evaluated under both ImageNet-pretrained and randomly initialized settings using Optuna-based hyperparameter tuning and stratified 5-fold cross-validation.

Under ImageNet-pretrained initialization, the supervised baseline achieves the best macro-F1 (0.852 ± 0.033), while SimCLR, VAE, and Mean Teacher remain competitive but consistently lower. Under random initialization, self-supervised pretraining becomes more beneficial, with SimCLR achieving the strongest performance (0.750 ± 0.046), outperforming the supervised-from-scratch baseline (0.714 ± 0.034).

Feature-space analysis further shows that self-supervised methods can produce more separable embeddings without necessarily improving downstream macro-F1. Overall, the results suggest that unlabeled learning is most beneficial when strong pretrained visual representations are unavailable, whereas ImageNet-pretrained supervised fine-tuning remains the most reliable strategy for maximizing macro-F1 in this benchmark.

1 Introduction

Since its introduction, deep learning has become the dominant approach for many automated image analysis tasks across a wide range of fields. Deep learning methods, and convolutional neural networks (CNNs) in particular, rely heavily on access to large amounts of data in order to learn robust visual representations and stable decision boundaries. Among the domains where deep learning has attracted significant attention is the medical field, especially for image classification tasks in which images are assigned to predefined diagnostic or pathological categories. However, the performance of these models strongly depends on the availability of annotated data, which is often limited in medical applications.

In supervised image classification, models are trained using pairs of images and expert-provided labels. Although this paradigm has achieved strong performance in many medical imaging tasks, obtaining labeled data remains a major challenge. Medical annotations are expensive and time-consuming because they require domain expertise, and in many cases only relatively small annotated datasets are available compared with the much larger collections of unlabeled medical images routinely generated in clinical and research settings.

To address this limitation, a large body of research has explored methods that leverage unlabeled images in addition to labeled data. Semi-supervised learning methods train models using labeled and unlabeled data simultaneously, often through consistency regularization or pseudo-labeling strategies [12, 22, 23]. In contrast, self-supervised learning methods first learn feature representations from unlabeled data alone and subsequently fine-tune the learned representations using labeled data [4, 11]. Prior work has shown that these approaches can improve image classification performance by extracting useful representations from large unlabeled datasets, particularly in settings where labeled data are scarce.

Building on these ideas, this work focuses on the classification of amyloid- β ($A\beta$) plaque types in brain pathology images. $A\beta$ plaques are protein aggregates associated with Alzheimer’s disease and appear in multiple morphological forms. Accurate plaque-type classification is therefore relevant for studying pathological heterogeneity and disease progression. This work builds on the dataset and preprocessing pipeline introduced by Moreira-Kanaley [14], which provides automatically segmented plaque crops together with a smaller expert-annotated subset. Their work proposed a supervised learning framework for plaque classification using a limited labeled dataset. However, despite the availability of millions of additional unlabeled plaque crops generated by the same segmentation pipeline, the impact of unlabeled data on this task was not investigated.

The main research question addressed in this work is: *can unlabeled plaque images improve classification performance in $A\beta$ plaque classification?*

To answer this question, we compare a supervised baseline against three semi-supervised methods (Pi-Model, FixMatch, and Mean Teacher) and two self-supervised learning strategies (SimCLR and a variational autoencoder) using the same labeled and unlabeled data pools. In addition, because pretrained initialization can strongly influence downstream representation learning, all experiments are conducted both with ImageNet-pretrained initialization and with random initialization from scratch.

The remainder of this report is organized as follows. Section 2 introduces the dataset, preprocessing steps, model architectures, and learning paradigms used in this work. Section 3 describes the experimental setup, hyperparameter optimization procedure, and evaluation protocol. Section 4 presents the experimental results together with feature-space analysis and discussion. Finally, Section 5 summarizes the main findings, while Section 6 discusses the limitations of the current study and directions for future work.

2 Methodology

In this section, we describe the methodology used in this work. We begin by introducing the dataset in Section 2.1. Section 2.2 then outlines the preprocessing steps applied to prepare the data for model training. Finally, Section 2.3 presents the learning paradigms considered (supervised, semi-supervised, and self-supervised), along with the corresponding models.

2.1 Dataset Description

The dataset used in this work is derived from amyloid- β ($A\beta$) plaque segmentations produced in prior work by Moreira-Kanaley [14]. It consists of 481 RGB whole-slide images (WSIs) acquired from four distinct cerebral regions of brain tissue collected at Amsterdam UMC. The WSIs originate from 93 healthy centenarians and 29 individuals diagnosed with Alzheimer’s disease (AD), as reported there.

In their work, each WSI was processed using a U-Net segmentation model [18], which is a convolutional neural network that produces pixel-level segmentations of the plaques. These segmentations, along with associated meta-data (including plaque roundness and area), were stored in HDF5 files, with one file per subject. This segmentation pipeline resulted in approximately 4.7 million segmented plaque images at varying resolutions. A small subset of 589 plaques was manually annotated by a neuropathology expert (S. K. Rhode) into the following classes:

- **Diffuse:** Softly contoured, cloud-like deposits with fuzzy borders and relatively light staining intensity [20, 24].
- **Cored:** Plaques with a dense central core and a well-defined outline, often surrounded by a halo of damaged neurites [20, 24].
- **Compact:** Small to medium-sized plaques appearing as tightly packed, uniformly bright structures with sharp boundaries [20, 24].
- **Coarse:** Larger, irregular plaques with a granular or “moth-eaten” appearance, characterized by multiple bright regions and internal voids [3].
- **CAA:** Linear or ring-like $A\beta$ deposits aligned with blood vessel walls, indicative of cerebral amyloid angiopathy [20, 24].
- **Subpial:** Band-like deposits located beneath the brain surface, forming a thin layer along the pial boundary [25].
- **OtherAB:** Plaques with valid bounding boxes that do not fit into the primary classes.
- **UndefAB:** Ambiguous $A\beta$ -positive plaques that cannot be reliably categorized.
- **NonAB:** Structures that do not exhibit $A\beta$ staining.

The first six categories are considered primary $A\beta$ plaque types, as they are well-defined in the neuropathology literature. The remaining three classes are used to filter ambiguous or non- $A\beta$ structures, improving the overall quality of the training data. Table 1 summarizes the class distribution in the annotated subset and the overall dataset composition, and Figure 1 shows representative examples of annotated plaques.

Table 1: Distribution of annotated plaque classes and overall dataset composition.

Annotated plaques ($n = 589$)	
Class	Count (%)
Diffuse	81 (13.7%)
Cored	107 (18.1%)
Compact	63 (10.6%)
Coarse	79 (13.4%)
CAA	74 (12.5%)
Subpial	73 (12.3%)
OtherAB	31 (5.2%)
UndefAB	43 (7.3%)
NonAB	38 (6.4%)
Unlabeled plaques ($n = 4,746,466$)	
Sampled	10,000
Total dataset size	10,589

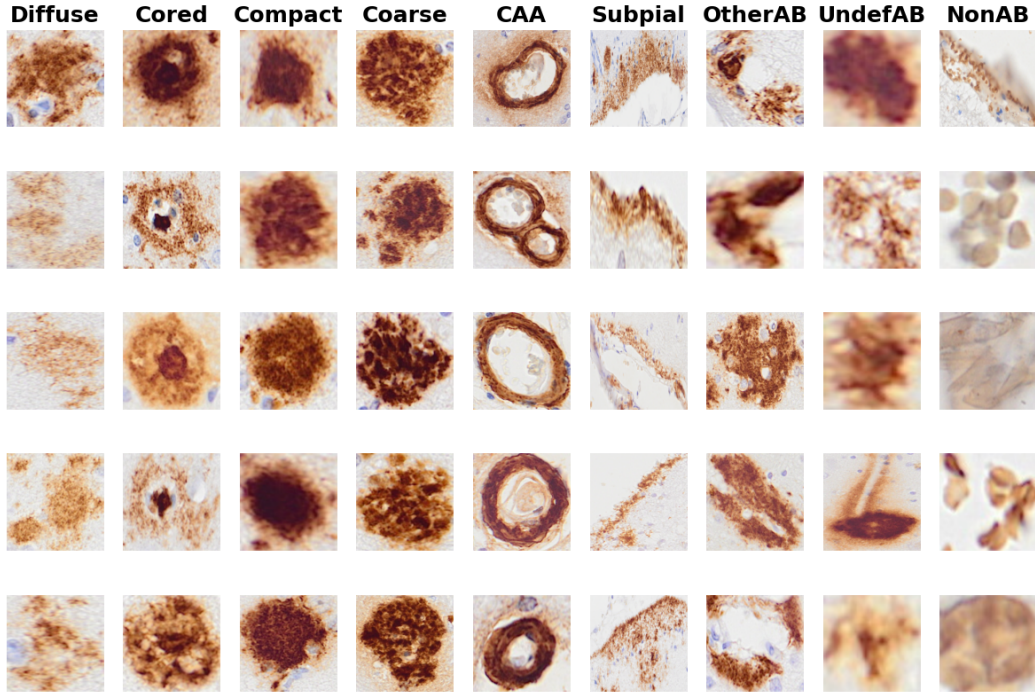


Figure 1: Examples of annotated plaques. For each $A\beta$ plaque class five examples are shown, each downsampled to 224×224 pixels.

2.2 Data Preprocessing

From the full dataset, we construct a working subset consisting of 589 annotated plaques and a random sample of 10,000 unlabeled plaques. This results in a total of 10,589 samples. Each plaque is represented as an RGB crop corresponding to its bounding box. These crops are extracted from HDF5 files and stored as PNG images.

Since the images have varying resolutions, all samples are resized to a fixed resolution of 224×224 pixels using bilinear interpolation. Although such downsampling may affect performance, particularly in low-data regimes, we adopt this standardized resolution to ensure a fair comparison across models. After resizing, images are normalized channel-wise using the ImageNet [7] mean and standard deviation ($[0.485, 0.456, 0.406]$ and $[0.229, 0.224, 0.225]$, respectively).

In addition to the image data, we extract two scalar morphological features, namely *roundness* and *area*, from the metadata stored in the HDF5 files. These features are stored alongside the image identifiers and labels, when available. They are standardized using z-score normalization based on statistics computed from the training set before being used as optional auxiliary inputs.

To improve robustness in the low-data regime, stochastic data augmentation is applied to labeled samples. For each labeled plaque, we generate *three* augmented views using transformations such as random horizontal and vertical flips, rotations, and color jitter. Further details are provided in Appendix A.2. Figure 2 illustrates the effect of normalization and augmentation.

For semi-supervised and self-supervised learning, additional augmentation pipelines are defined for unlabeled data. Specifically, each unlabeled image is transformed into two augmentations that support the semi-supervised and self-supervised learning objectives. These augmentations are required by the consistency-based learning methods described in Section 2.3. Detailed augmentation strategies and hyperparameters are provided in Appendix A.2.

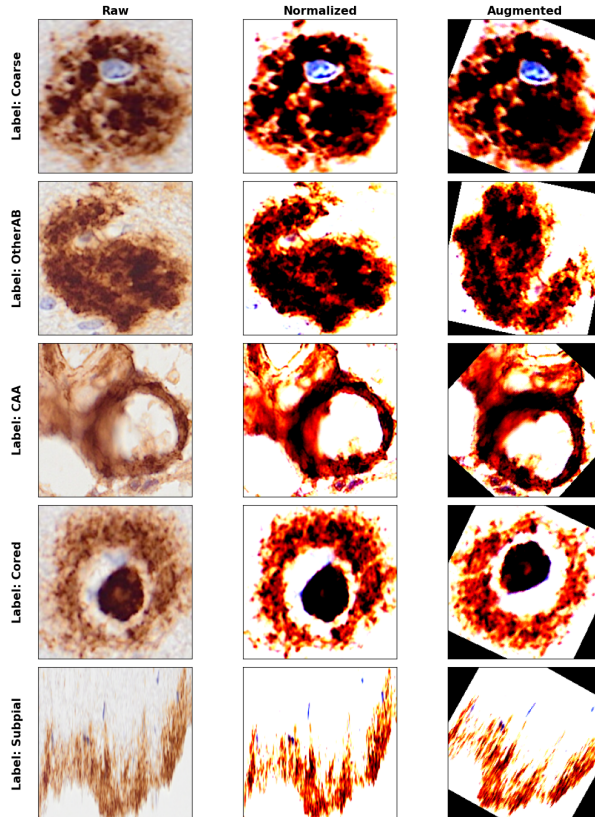


Figure 2: Illustration of normalization and augmentation for $A\beta$ plaque images. Left: raw image. Middle: normalized image. Right: an example augmented view.

2.3 Models and Learning Paradigms

The objective of this work is to classify amyloid- β plaques into the nine classes described in Section 2.1. To this end, we consider three learning paradigms: supervised, semi-supervised, and self-supervised. For each paradigm, we evaluate a set of corresponding models.

Section 2.3.1 introduces the shared feature extractor architecture used as a common backbone across all models. Section 2.3.2 then describes the supervised baseline, which serves as a reference for comparison. Section 2.3.3 presents the semi-supervised framework used to incorporate unlabeled data into the training objective. Finally, Section 2.3.4 describes the self-supervised framework, in which the feature extractor is pretrained on unlabeled data and subsequently fine-tuned on labeled data.

2.3.1 Model Architecture

All learning paradigms considered in this work share a common backbone architecture for feature extraction. This design choice ensures consistency and comparability across experimental settings. Specifically, we employ a **ResNet-18** [10] backbone initialized with ImageNet-pretrained weights [7], which provide strong general-purpose visual features and improve convergence in the low-data regime.

The original classification layer of ResNet-18 is replaced with a projection layer followed by a dropout layer. The projection layer is implemented as a linear transformation that reduces the feature dimensionality and produces more compact representations. The dropout layer is applied to regularize the learned features and reduce the risk of overfitting.

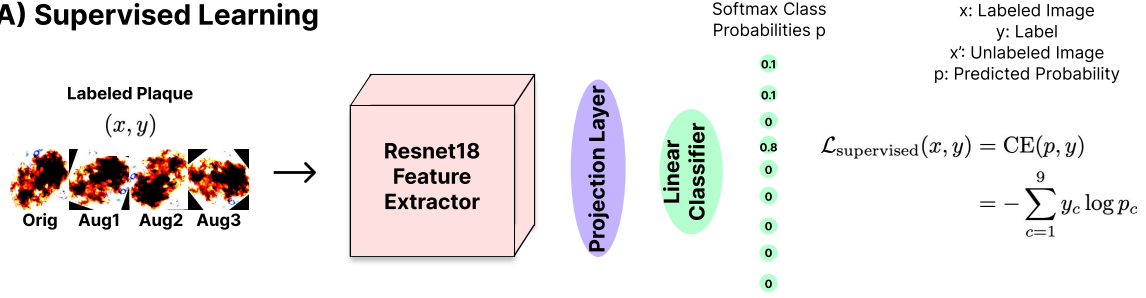
To mitigate possible overfitting in the low-data regime, we use a hyperparameter to control whether a subset of the backbone layers is frozen during training. If this hyperparameter is enabled, the first n blocks (a hyperparameter itself) of the network are frozen to preserve pretrained features. At a later stage, selected blocks are unfrozen after a predefined number of epochs. This strategy balances stability and adaptability and allows the model to specialize without overfitting in early training stages. If it is disabled, all backbone layers are trained.

The resulting feature representations are passed to a linear classifier head that maps them to the nine target plaque classes. This feature extractor and classifier combination is used consistently across all learning paradigms. As a result, differences in performance can be attributed primarily to the learning strategy rather than architectural variations.

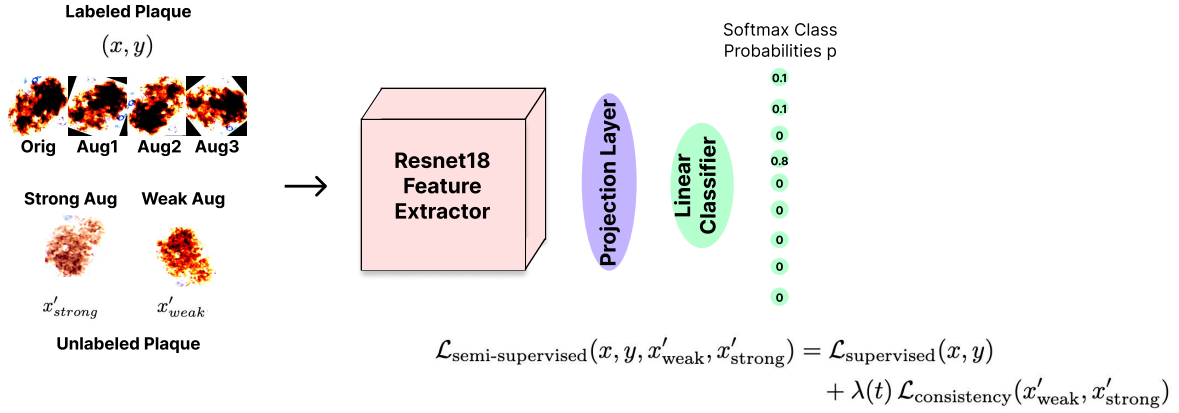
The training procedure differs across paradigms in how labeled and unlabeled data are utilized. In the supervised setting, the model is trained using only labeled data with the standard multi-class cross-entropy loss. In the semi-supervised setting, unlabeled data are incorporated through a consistency regularization objective in addition to the supervised loss. In the self-supervised setting, the feature extractor is first pretrained on unlabeled data using a self-supervised objective. After pretraining, the backbone and classifier head are fine-tuned on labeled data. More details for each learning paradigm are provided in the following sections.

Figure 3 provides an overview of these differences and highlights how each paradigm leverages labeled and unlabeled data during training.

A) Supervised Learning



B) Semi-Supervised Learning



C) Self-Supervised Learning

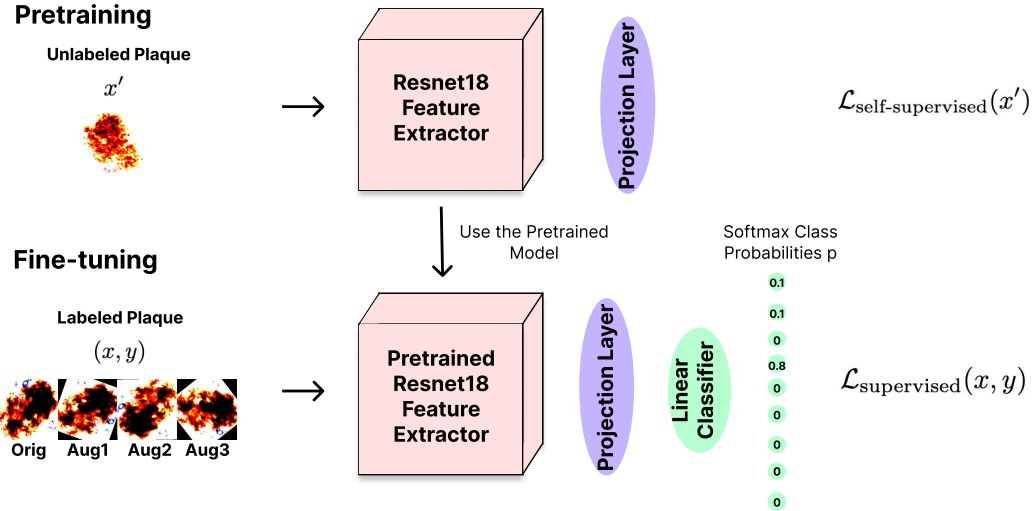


Figure 3: Overview of the training procedures across learning paradigms. (A) Supervised learning uses only labeled plaques, where augmented inputs are processed to compute the cross-entropy loss. (B) Semi-supervised learning combines labeled and unlabeled data. Labeled samples contribute to the supervised loss, while unlabeled samples are transformed into weakly and strongly augmented views to compute a consistency loss. The total loss is controlled by a time-dependent weighting factor. (C) Self-supervised learning first pretrains the feature extractor using unlabeled data with a self-supervised objective. The model is then fine-tuned on labeled data for classification.

2.3.2 Supervised Learning Baseline

The supervised baseline serves as a reference model that is trained exclusively on labeled plaque images. It provides a performance benchmark that is used to evaluate the benefit of incorporating unlabeled data in more advanced learning paradigms.

As illustrated in Figure 3, the model consists of a feature extractor backbone followed by a linear classifier head that produces class logits. The model is optimized using the standard multi-class cross-entropy loss, which encourages correct class assignment for each labeled sample.

For a single input x with ground-truth label y , the loss function is defined as

$$\mathcal{L}_{\text{supervised}}(x, y) = \text{CE}(p, y) \quad (1)$$

$$= - \sum_{c=1}^9 y_c \log p_c, \quad (2)$$

where $\text{CE}(\cdot, \cdot)$ denotes the multi-class cross-entropy loss, p_c represents the predicted probability that input x belongs to class c , and y_c is the corresponding one-hot encoded target.

The predicted probabilities p_c are obtained by applying the softmax function to the output logits of the classifier head. This formulation penalizes incorrect predictions and assigns higher loss values when the model is confident in an incorrect class.

All hyperparameters for the supervised baseline and their selected values are listed in Appendix A.3.

2.3.3 Semi-Supervised Learning

The semi-supervised approach extends the supervised baseline by incorporating unlabeled data through consistency regularization. This paradigm is particularly suitable for the present setting, where labeled data are limited compared to the large pool of unlabeled plaque images.

During training, each iteration processes both a batch of labeled samples and a batch of unlabeled samples. For labeled data, the objective remains the standard cross-entropy loss used in the supervised baseline.

For unlabeled data, the model enforces prediction consistency under different augmentations of the same input. Each unlabeled image is transformed into weakly and strongly augmented views (see Appendix A.2 for augmentation details), which are processed independently by the model.

The total training objective is defined as

$$\begin{aligned} \mathcal{L}_{\text{semi-supervised}}(x, y, x'_{\text{weak}}, x'_{\text{strong}}) &= \mathcal{L}_{\text{supervised}}(x, y) \\ &+ \lambda(t) \mathcal{L}_{\text{consistency}}(x'_{\text{weak}}, x'_{\text{strong}}), \end{aligned} \quad (3)$$

where x and y denote labeled inputs and their corresponding labels, and x'_{weak} and x'_{strong} denote the weakly and strongly augmented views of unlabeled samples. The function $\lambda(t)$ is a time-dependent weighting factor that controls the contribution of the consistency loss.

The ramp-up schedule for $\lambda(t)$ improves training stability in the early stages. At the beginning of training, model predictions on unlabeled data are often unreliable. A high consistency weight at this stage can negatively affect learning. The gradual increase of $\lambda(t)$ allows the model to first learn stable representations from labeled data. The model then incorporates information from unlabeled data as training progresses.

Different semi-supervised methods vary primarily in how the consistency loss $\mathcal{L}_{\text{consistency}}$ is defined. Common choices include mean squared error, Kullback–Leibler divergence, and cross-entropy between predicted probability distributions of the weakly and strongly augmented views.

The hyperparameters for the semi-supervised methods and their selected values are listed in Appendix A.3.

Pi-Model The Pi-Model [12] enforces consistency between predictions obtained from different augmentations of the same unlabeled image. For each unlabeled sample, the model processes both a weakly augmented view and a strongly augmented view, which produce two probability distributions over the target classes. The strong augmentation applies more severe transformations than the weak augmentation and introduces greater variation in appearance.

The consistency loss penalizes discrepancies between the two predictions and encourages the model to learn representations that remain stable under input perturbations. This approach does not rely on pseudo-labels. Instead, it directly aligns the predicted distributions using a distance metric such as mean squared error or Kullback–Leibler divergence. Moreover, Pi-Model does not require additional architectural components or auxiliary networks and it serves as a simple and effective baseline for consistency-based semi-supervised learning.

Figure 4 provides an overview of the consistency loss computation in the Pi-Model framework.

Pi-Model

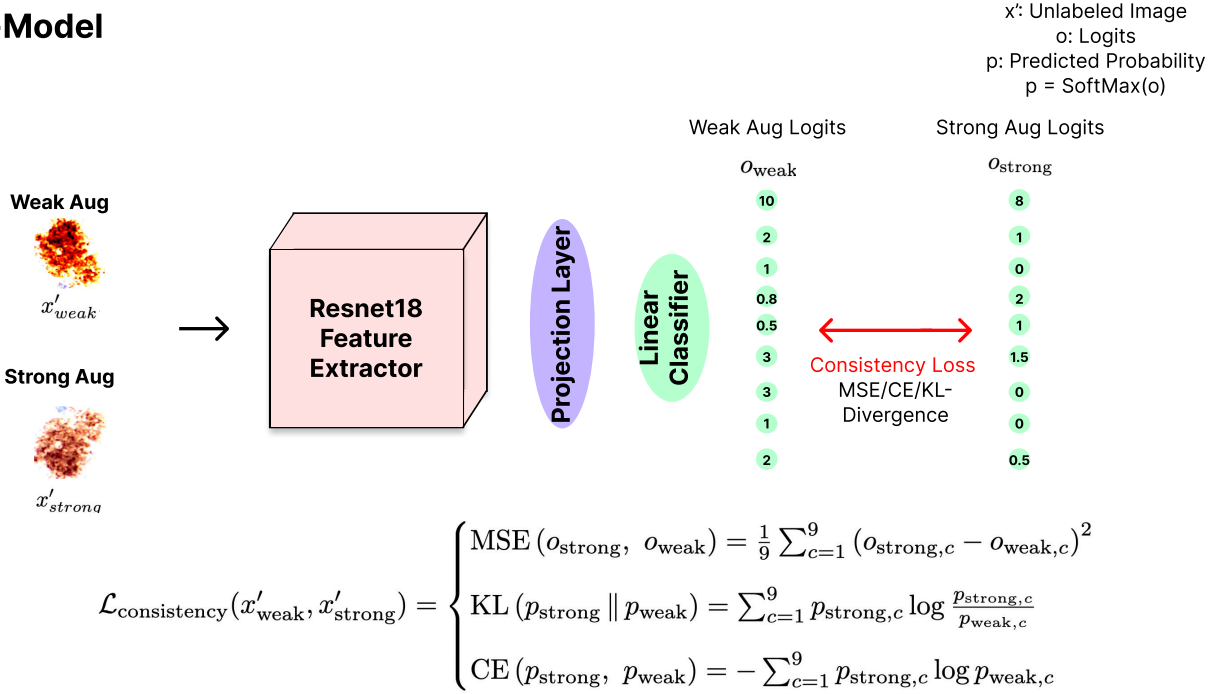


Figure 4: Semi-supervised Pi-Model consistency loss computation. An unlabeled input image is transformed into weakly and strongly augmented views. Both views are processed by a shared network to produce class probability distributions. A consistency loss enforces agreement between these predictions and promotes invariance to input perturbations. Abbreviations: MSE = Mean Squared Error; KL = Kullback–Leibler Divergence; CE = Cross-Entropy.

FixMatch FixMatch [22] combines pseudo-labeling with consistency regularization to leverage unlabeled data in a selective and reliable manner. For each unlabeled image, the model first processes a weakly augmented view to obtain class probabilities.

If the maximum predicted probability exceeds a predefined confidence threshold, the corresponding class is selected as a pseudo-label. The pseudo-label is represented as a one-hot probability distribution, where the probability for class c is defined as

$$p_{pseudo,c} = \begin{cases} 1 & \text{if } c = \text{argmax}(p_{weak}), \\ 0 & \text{otherwise,} \end{cases} \quad (4)$$

where p_{weak} denotes the predicted class probabilities for the weakly augmented view.

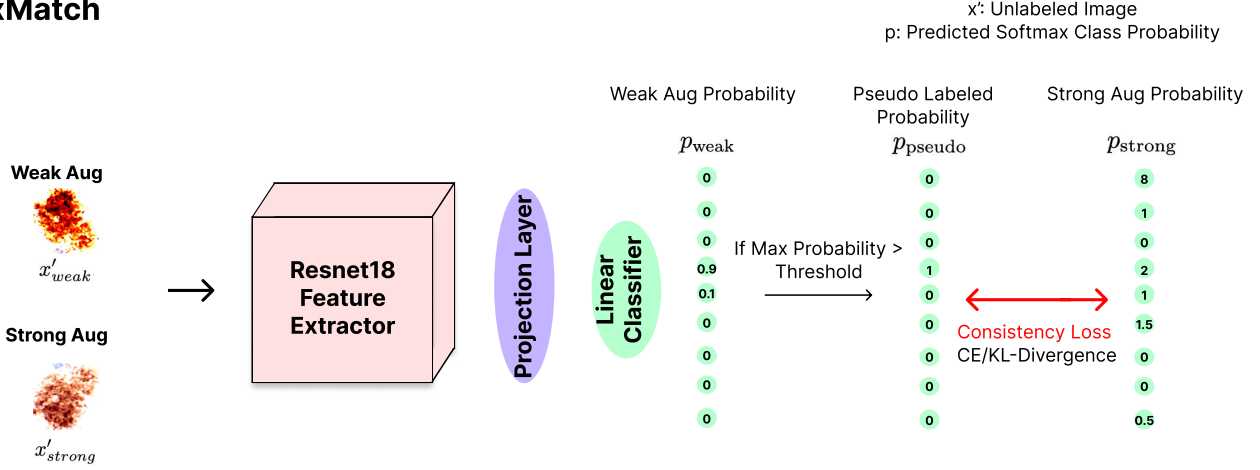
The strongly augmented view of the same image is then processed by the model. A consistency loss, such as cross-entropy or Kullback–Leibler divergence, is computed between the predictions of the strongly augmented view and the pseudo-label distribution.

This mechanism ensures that only high-confidence predictions contribute to the unsupervised loss. It reduces the impact of noisy or incorrect pseudo-labels and improves training stability. The confidence threshold therefore

plays a critical role in balancing data utilization and label reliability.

Figure 5 provides an overview of the FixMatch framework consistency loss computation.

FixMatch



$$\mathcal{L}_{\text{consistency}}(x'_{\text{weak}}, x'_{\text{strong}}) = \begin{cases} \text{KL}(p_{\text{strong}} \| p_{\text{pseudo}}) = \sum_{c=1}^9 p_{\text{strong},c} \log \frac{p_{\text{strong},c}}{p_{\text{pseudo},c}} \\ \text{CE}(p_{\text{strong}}, p_{\text{pseudo}}) = -\sum_{c=1}^9 p_{\text{strong},c} \log p_{\text{pseudo},c} \end{cases}$$

Figure 5: Semi-supervised FixMatch consistency loss computation. An unlabeled input image is transformed into weakly and strongly augmented views. The weakly augmented view is used to produce class probabilities. If the maximum predicted probability exceeds a predefined confidence threshold, a pseudo-label is generated as a one-hot distribution. The strongly augmented view is processed independently, and a consistency loss is computed between its predicted probabilities and the pseudo-label. This design ensures that only high-confidence predictions contribute to the unsupervised loss, which reduces the effect of incorrect pseudo-labels.

Mean Teacher The Mean Teacher framework [23] introduces a teacher-student architecture to improve training stability and generalization. The student model corresponds to the primary network that is optimized during training. The teacher model is defined as an exponential moving average (EMA) of the student parameters.

After each training step, the teacher parameters are updated according to

$$\theta_t^{(\text{teacher})} = \alpha \theta_{t-1}^{(\text{teacher})} + (1 - \alpha) \theta_t^{(\text{student})}, \quad (5)$$

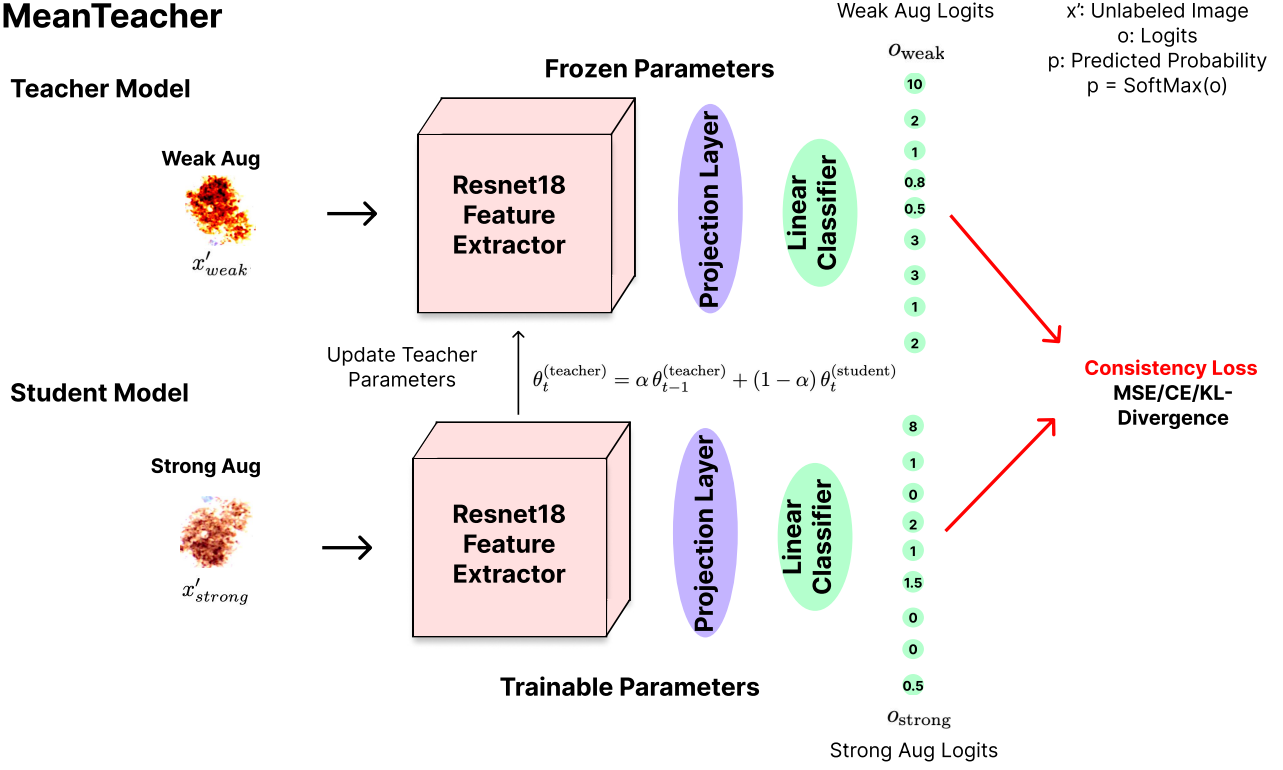
where $\alpha \in [0, 1)$ is the EMA decay rate. Larger values of α assign more weight to past teacher parameters and result in slower updates, while smaller values allow the teacher to adapt more quickly to the student.

During training, the student processes the strongly augmented view of each unlabeled image, and the teacher processes the weakly augmented view. A consistency loss is then applied between their predictions. This loss encourages the student to align with the teacher, which provides a more stable target due to the smoothing effect of the EMA updates. After training at inference time, the teacher model is used for prediction because it typically produces more reliable outputs than the student.

The EMA mechanism smooths the parameter trajectory across training iterations. As a result, the teacher model captures a more stable representation of the learned function. This property improves generalization and reduces sensitivity to noise in individual updates, especially under the strong augmentation applied to unlabeled data.

Figure 6 illustrates the consistency loss computation in the Mean Teacher framework.

MeanTeacher



$$\mathcal{L}_{\text{consistency}}(x'_{\text{weak}}, x'_{\text{strong}}) = \begin{cases} \text{MSE}(o_{\text{strong}}, o_{\text{weak}}) = \frac{1}{9} \sum_{c=1}^9 (o_{\text{strong},c} - o_{\text{weak},c})^2 \\ \text{KL}(p_{\text{strong}} \parallel p_{\text{weak}}) = \sum_{c=1}^9 p_{\text{strong},c} \log \frac{p_{\text{strong},c}}{p_{\text{weak},c}} \\ \text{CE}(p_{\text{strong}}, p_{\text{weak}}) = - \sum_{c=1}^9 p_{\text{strong},c} \log p_{\text{weak},c} \end{cases}$$

Figure 6: Semi-supervised Mean Teacher consistency loss computation. A student model is optimized using both labeled and unlabeled data. A teacher model is maintained as an exponential moving average of the student parameters. For each unlabeled input, the student and teacher process different augmented views. A consistency loss is applied between their predictions, which encourages the student to align with the more stable teacher outputs.

2.3.4 Self-Supervised Learning

Self-supervised learning refers to a class of methods that learn useful data representations from large amounts of unlabeled data by optimizing a surrogate objective. These objectives are designed to create supervisory signals directly from the data itself, for example through contrastive learning.

In this work, the feature extractor backbone is first pretrained on unlabeled plaque images using a self-supervised objective. After pretraining, a classifier head is attached, and the entire model is fine-tuned on labeled data using a supervised loss.

This training procedure differs from semi-supervised learning in how labeled and unlabeled data are utilized. In self-supervised learning, the model is first trained exclusively on unlabeled data. The pretrained model is then adapted to the downstream classification task using labeled data. In contrast, semi-supervised learning optimizes a joint objective that combines labeled and unlabeled data within a single training phase.

The two-stage training process in self-supervised learning allows the model to learn general-purpose visual representations before adapting to the specific plaque classification task. This approach is particularly beneficial in low-data regimes, where labeled samples are limited.

Before introducing the specific self-supervised methods considered in this work, we note that all pretraining hyperparameters and their selected values are reported in Appendix A.3.

Contrastive Learning (SimCLR-Style) Following SimCLR [4], contrastive learning trains representations that are invariant to input perturbations while remaining discriminative across different images. This is achieved by maximizing agreement between different augmented views of the same image while minimizing agreement between representations of distinct images.

For each input image, two strong stochastic augmentations are generated. Each view is processed by a shared feature extractor followed by a projection head, which maps representations into a latent space where the contrastive objective is applied. The model is trained such that representations of the same image (positive pairs) are pulled closer together, while representations of different images (negative pairs) are pushed apart.

Let x'_i and x'_j denote two augmented views of the same image, and let x'_k denote all other augmented views in the batch. Let z_i and z_j be the outputs of the feature extractor plus projection head applied to x'_i and x'_j , respectively. The SimCLR contrastive loss for a positive pair is defined as:

$$\mathcal{L}_{i,j} = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k=1, k \neq i}^{2N} \exp(\text{sim}(z_i, z_k)/\tau)} \quad (6)$$

where $\text{sim}(\cdot, \cdot)$ denotes cosine similarity and τ is a temperature parameter controlling the sharpness of the similarity distribution. The batch contains N original images and $2N$ augmented views in total.

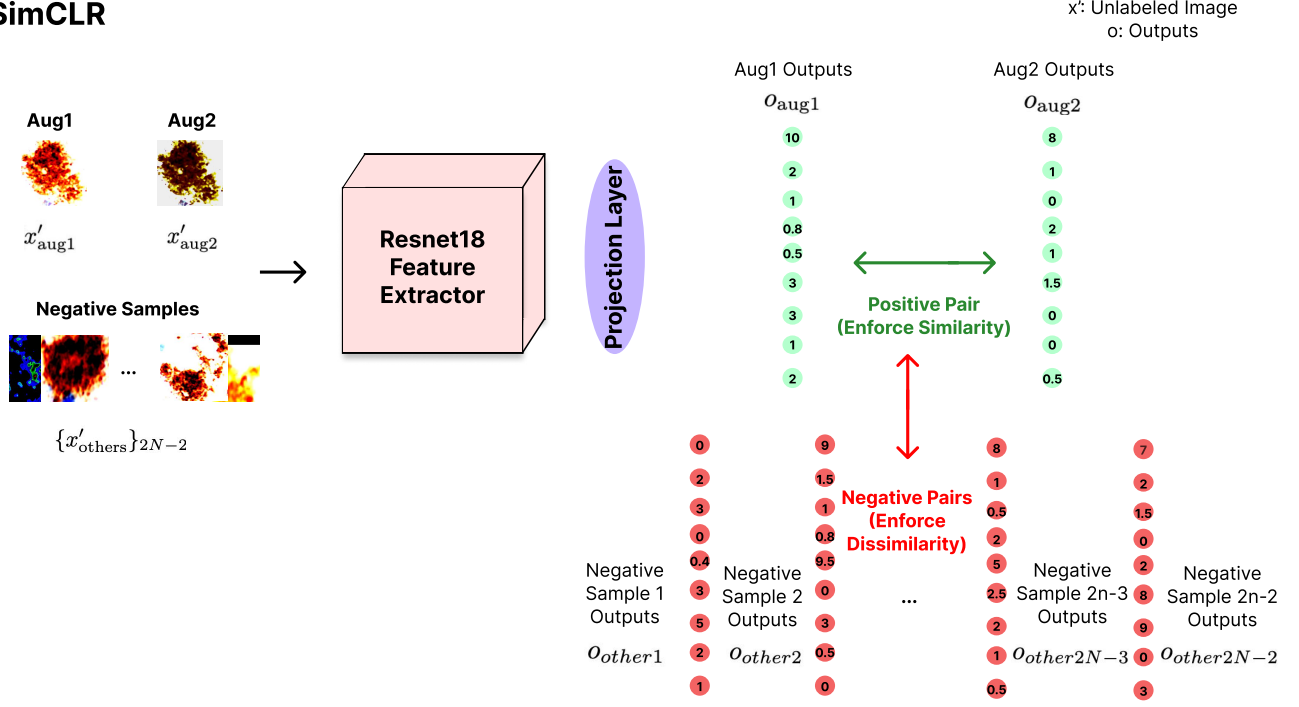
The denominator includes all views in the batch except the anchor view, which are treated as negative samples. This formulation promotes similarity in the embedding space between positive pairs, that is, two augmented views of the same image, while simultaneously reducing similarity with negative samples, meaning all other augmented views in the batch.

The temperature parameter τ controls the concentration of the similarity distribution. Lower values of τ increase the emphasis on hard negatives, while higher values produce smoother similarity distributions.

After pretraining, a linear classifier is attached to the pretrained backbone and trained on labeled data using the standard cross-entropy loss.

Figure 7 provides an overview of the pretraining step of the SimCLR framework.

SimCLR



$$\mathcal{L}_{\text{contrast}}(x'_{aug1}, x'_{aug2}, \{x'_{others}\}_{2N-2}) = -\log \frac{\exp(\text{sim}(o_{aug1}, o_{aug2})/\tau)}{\sum_{o \in \{o_{aug2}\} \cup \{o_{others}\}} \exp(\text{sim}(o_{aug1}, o)/\tau)}$$

Figure 7: SimCLR contrastive learning framework pretraining step. Each input image is transformed into two augmented views, which form a positive pair. All other augmented views in the batch act as negative samples. The representations are passed through a projection head, and a contrastive loss is applied to maximize similarity between positive pairs and minimize similarity with negative samples. For a batch of N images, there are $2N$ augmented views in total, and each view is contrasted against the remaining $2N - 1$ views.

Variational Autoencoder (VAE) As an alternative self-supervised approach, variational autoencoders (VAEs) [11] are used to learn latent representations of plaque images. Like standard autoencoders, VAEs consist of an encoder, which maps the input image to a latent representation, and a decoder, which maps the latent representation back to the input space. However, in a VAE the latent representation is not a single vector but a distribution over latent variables. The encoder maps each input image to the parameters of a latent Gaussian distribution, from which a latent variable is sampled and passed to the decoder to reconstruct the input. VAEs are particularly useful in this setting because they learn a compact and continuous latent space that captures meaningful morphological variation between plaque types. This property supports downstream classification in low-label regimes and facilitates exploration of the underlying data structure, including detection of atypical or ambiguous plaque morphologies.

The VAE training objective consists of a reconstruction term and a regularization term that enforces the latent distribution to follow a predefined prior, in this case a standard normal distribution. This objective is defined as:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{recon}} + \beta \mathcal{L}_{\text{KL}}, \quad (7)$$

where β controls the trade-off between reconstruction quality and latent space regularization. The reconstruction term encourages faithful reconstruction of the input, while the KL term regularizes the latent space by encouraging it to follow a structured prior distribution.

The reconstruction loss is defined as:

$$\mathcal{L}_{\text{recon}} = \frac{1}{N} \sum_{i=1}^N \|x_i - \hat{x}_i\|^2, \quad (8)$$

where x_i is the input image and $\hat{x}_i$ is the reconstructed image.

The KL divergence term measures the difference between the approximate posterior distribution $q(z|x)$ and the prior distribution $p(z)$ in the latent space. Here, $q(z|x)$ denotes the encoder-defined distribution over latent variables z conditioned on the input image x , while $p(z)$ is the prior distribution over latent variables, here chosen as a standard normal distribution $\mathcal{N}(0, I)$.

The Kullback–Leibler divergence is defined as:

$$\mathcal{L}_{\text{KL}} = \frac{1}{N} \sum_{i=1}^N D_{\text{KL}}(q(z|x_i) \| p(z)) = -\frac{1}{2N} \sum_{i=1}^N \sum_{j=1}^d (1 + \log \sigma_{ij}^2 - \mu_{ij}^2 - \sigma_{ij}^2), \quad (9)$$

where d denotes the dimensionality of the latent space, and μ_{ij} and σ_{ij}^2 are the mean and variance parameters produced by the encoder for latent dimension j of sample i .

In our implementation, for the encoder we use the ResNet-18 backbone followed by two separate linear layers that map pooled features to the mean vector $\boldsymbol{\mu}$ and the log-variance vector $\log \boldsymbol{\sigma}^2$ of a diagonal Gaussian in $\mathbb{R}^d$, where d is the latent dimension (a tunable hyperparameter together with the KL weight β and the choice of reconstruction loss: mean squared error, ℓ_1 , or binary cross-entropy between $\hat{x}$ and the normalized input).

The decoder maps a sampled latent vector $\mathbf{z} \in \mathbb{R}^d$ back to an RGB image of the same spatial size as the input. First, a fully connected layer expands $\mathbf{z}$ into a one-dimensional vector that is reshaped into a low-resolution feature map; its height and width are chosen as one-sixteenth of the input resolution so that, after a fixed stack of four transposed convolutions—each using kernel size 4, stride 2, and padding 1 so that height and width double at every stage—the spatial extent matches the input while the channel width is reduced until the output has three channels. Rectified linear units are applied after each transposed convolution except the last, whose output is left unconstrained so that reconstructions are real-valued maps matched to normalized RGB targets by the reconstruction loss.

After pretraining, the decoder and the Gaussian-parameter heads are discarded; only the ResNet-18 backbone weights are transferred to initialize the supervised model, and a linear classifier is trained on top of the pretrained feature extractor using the standard cross-entropy loss.

An overview of the pretraining step of the VAE framework is provided in Figure 8.

VAE

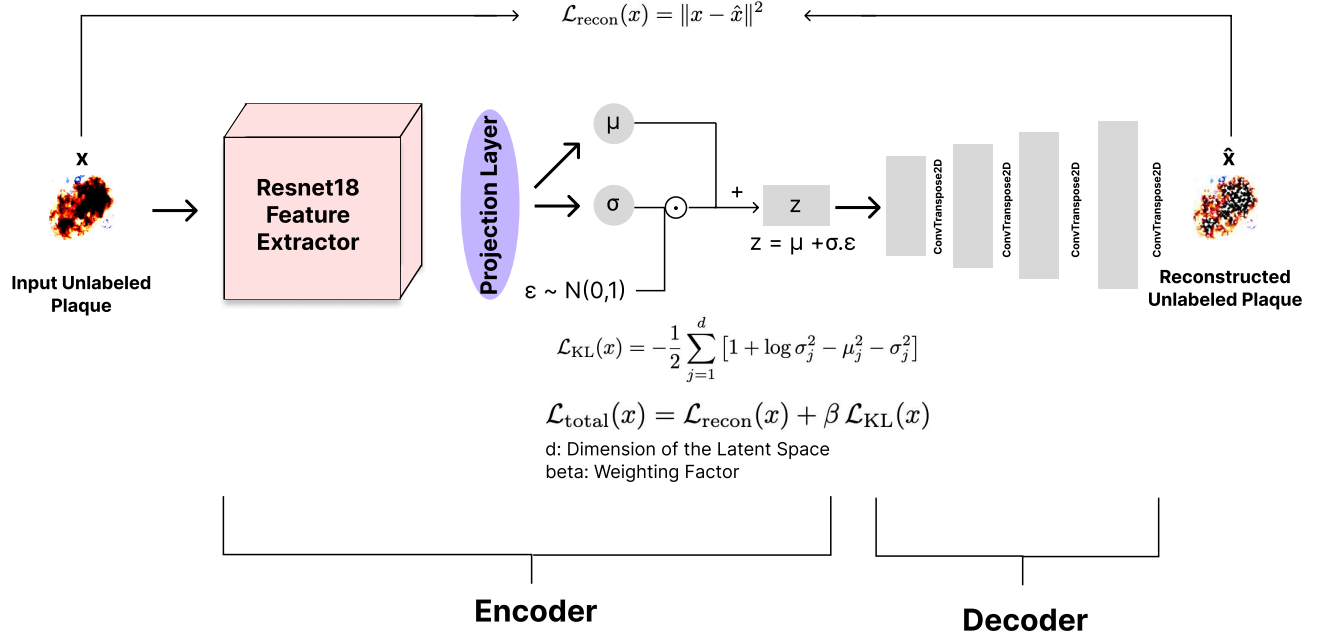


Figure 8: Variational Autoencoder (VAE) framework pretraining step. An input image is encoded into the parameters of a latent Gaussian distribution, from which a latent sample is drawn and decoded to reconstruct the input. The model is trained using a combination of reconstruction loss and KL divergence, which encourages the latent space to follow a standard normal prior.

3 Experimental Setup

This section summarizes the core protocol used to evaluate the supervised, semi-supervised, and self-supervised models introduced in Section 2.3. Low-level implementation details (software stack, hardware profile, and full reproducibility notes) are moved to Appendix A.1 to keep the main narrative focused on the experimental logic.

3.1 Configuration Protocol

All experiments are controlled through hierarchical JSON configuration files. Shared settings, including dataset configuration, optimization defaults, and backbone architecture parameters, are defined in a general configuration file. Paradigm-specific configurations are defined in separate files corresponding to supervised, semi-supervised, and self-supervised training.

Search spaces are declared explicitly in the configuration files in the `hyperparameter_tuning` blocks. These blocks map directly to the hyperparameter tables in Section 2.3, ensuring traceability between reported results and executable configuration. This configuration setup allows for reproducibility and enables easy experimentation with different models and training paradigms. For an example of the configuration file format check Appendix A.1.

3.2 Hyperparameter Optimization Protocol

Hyperparameters are optimized with Optuna’s Tree-structured Parzen Estimator (TPE) sampler [1]. TPE is a popular sampler for hyperparameter optimization and is known for its efficiency and effectiveness in hyperparameter tuning tasks. It is a Bayesian optimization algorithm that samples hyperparameters based on the posterior

distribution of the hyperparameters. For each trial, candidate values are sampled from the predefined search spaces, injected into the runtime configuration, and evaluated via full model training.

For each method, 20 trials (combinations of hyperparameters) are executed. Trial quality is scored using 5-fold cross-validation macro-F1, and the best trial is selected for final reporting. Use of TPE sampler was necessary because in this project the hyperparameter search space is large and complex, and we wanted to use a sampler that is efficient and effective in finding the best hyperparameters.

Complete run metadata are provided in Appendix A.3.

4 Results and Discussion

After hyperparameter tuning, each method is evaluated using the same stratified 5-fold cross-validation protocol on the full labeled subset (Section 3.2). For each method, we run two parallel experiments: one initialized with ImageNet-pretrained ResNet-18 [10, 7] weights and one trained from random initialization. Table 2 reports the average macro-F1 score (mean $\pm$ standard deviation across folds) for both initialization settings.

Macro-F1 is defined as the unweighted average of per-class F1 scores, which assigns equal importance to each class regardless of class frequency:

$$\text{Macro-F1} = \frac{1}{C} \sum_{i=1}^C F_i \quad (10)$$

where C is the number of classes and F_i is the F1 score for class i . The class-wise F1 score is defined as

$$F_i = 2 \cdot \frac{\text{Precision}_i \cdot \text{Recall}_i}{\text{Precision}_i + \text{Recall}_i}, \quad (11)$$

with

$$\text{Precision}_i = \frac{\text{TP}_i}{\text{TP}_i + \text{FP}_i}, \quad \text{Recall}_i = \frac{\text{TP}_i}{\text{TP}_i + \text{FN}_i}, \quad (12)$$

where TP_i , FP_i , and FN_i denote true positives, false positives, and false negatives for class i , respectively.

Table 2: Comparison of hyperparameter-tuned performance across all learning paradigms and methods (ResNet-18 backbone) under two initialization settings: ImageNet-pretrained weights and random initialization. Each value reports mean macro-F1 $\pm$ standard deviation across cross-validation folds. Macro-F1 assigns equal importance to all classes, while the standard deviation reflects stability across folds. The best-performing score across all methods and initialization settings is highlighted in gray.

Learning paradigm	Method	Macro-F1 score	
		Random initialization	ImageNet pretrained
Supervised	Supervised baseline	0.714 $\pm$ 0.034	0.852 $\pm$ 0.033
Semi-supervised	Pi-Model	0.715 $\pm$ 0.036	0.806 $\pm$ 0.034
Semi-supervised	FixMatch	0.725 $\pm$ 0.061	0.801 $\pm$ 0.016
Semi-supervised	Mean Teacher	0.725 $\pm$ 0.035	0.832 $\pm$ 0.017
Self-supervised	SimCLR pretraining + fine-tuning	0.750 $\pm$ 0.046	0.835 $\pm$ 0.054
Self-supervised	VAE pretraining + fine-tuning	0.735 $\pm$ 0.037	0.834 $\pm$ 0.047

4.1 Random initialization results

Under random initialization, the **SimCLR self-supervised method** achieves the highest macro-F1 score (**0.750 $\pm$ 0.046**), followed by the VAE-based self-supervised method (0.735 $\pm$ 0.037). Among the semi-supervised methods, Mean Teacher and FixMatch are tied at 0.725 (with different variances), both outperforming the supervised-from-scratch baseline (0.714 $\pm$ 0.034) and Pi-Model (0.715 $\pm$ 0.036).

These results suggest that unlabeled objectives are beneficial under random initialization, with the clearest gains coming from two-stage self-supervised pretraining. At the same time, the larger standard deviations observed for several unlabeled methods (especially FixMatch and SimCLR) indicate higher sensitivity across folds than the supervised baseline, likely due to stronger dependence on augmentation policies and objective-weight interactions.

4.2 ImageNet-pretrained setting results

In the ImageNet-pretrained setting, the **supervised baseline** achieves the highest macro-F1 score (0.852 ± 0.033). The closest methods are SimCLR (0.835 ± 0.054), VAE (0.834 ± 0.047), and Mean Teacher (0.832 ± 0.017), while Pi-Model (0.806 ± 0.034) and FixMatch (0.801 ± 0.016) remain clearly lower.

In short, ImageNet-pretrained weights [7] already give a strong starting representation, so adding unlabeled-data losses does not move macro-F1 very much on average. That said, the gap is small for the best runs: SimCLR, VAE, and Mean Teacher finish within about two macro-F1 points of the supervised baseline, while Pi-Model and FixMatch fall further behind.

Compared with joint semi-supervised optimization, two-stage self-supervised pretraining remains more consistently competitive in this setting. A plausible explanation is that pretraining and classification fine-tuning are decoupled, which reduces the conflict between supervised and unsupervised objectives when the starting representation is already strong.

4.3 Feature space analysis

To further analyze the quality of the learned feature representations in the ImageNet-pretrained setting, we extract feature vectors for the labeled samples from each method and visualize the first two principal components in Figure 9. We also compute silhouette scores for both the first two PCA dimensions and the full feature space, reported in Table 3.

The silhouette coefficient [19] measures how well samples cluster within their own class compared with neighboring classes:

$$\text{Silhouette score} = \frac{b - a}{\max(a, b)} \quad (13)$$

where a denotes the average intra-class distance and b denotes the distance to the nearest different class. Values lie in the range $[-1, 1]$, where higher values indicate better cluster separation. The silhouette scores therefore provide an indication of how well the learned feature representations are separated in the embedding space.

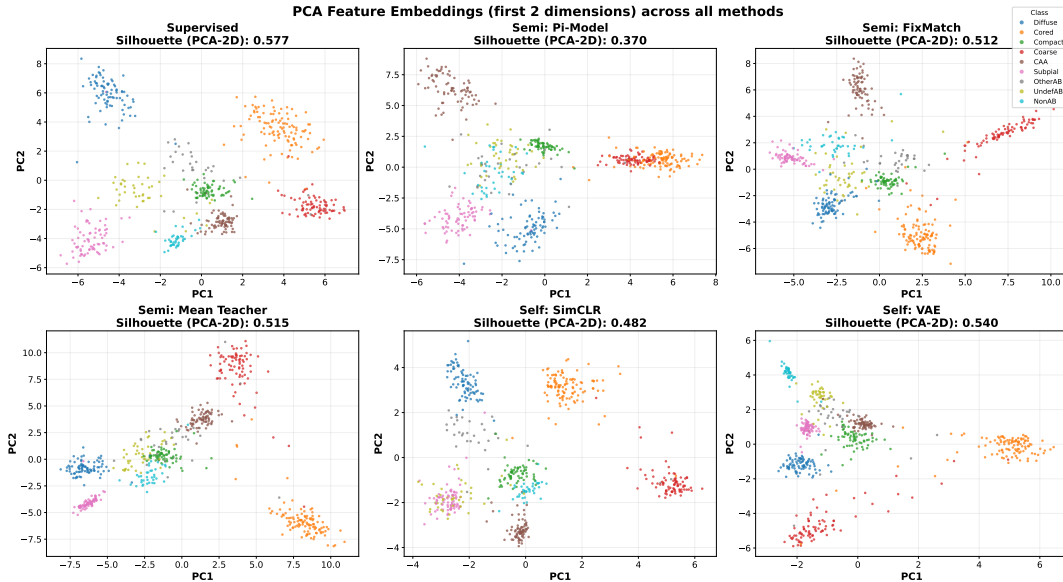


Figure 9: Visualization of the first two principal components of the feature representations for each method. Each subplot corresponds to a different method, and points are colored according to class label. These projections are only a qualitative view of geometry in two dimensions. The corresponding 2D PCA silhouette scores in Table 3 are highest for the supervised baseline, so stronger visual separation here is not guaranteed for semi- or self-supervised methods; full-dimensional silhouette scores tell a different ranking.

Table 3: Silhouette scores computed from feature representations of labeled samples. Scores are reported for both the first two PCA dimensions and the full feature space. The feature dimensionality reflects the projection size selected during hyperparameter tuning.

Model	Silhouette (PCA 2D)	Silhouette (all dims)	Feature Dim
Supervised	0.577	0.613	64
Semi: Pi-Model	0.370	0.578	64
Semi: Mean Teacher	0.515	0.662	128
Semi: FixMatch	0.512	0.642	64
Self: VAE	0.540	0.700	32
Self: SimCLR	0.482	0.690	32

The feature-space analysis reveals a different trend from the classification results. The silhouette analysis shows a mixed ranking: the supervised baseline achieves the highest score in the 2D PCA projection, whereas VAE and SimCLR achieve the highest silhouette scores in the full-dimensional feature space. Mean Teacher and FixMatch also exceed the supervised baseline in the full feature space.

This suggests that the full feature spaces learned by the self-supervised methods are more separable than those of the supervised baseline, even though the supervised baseline still achieves the highest macro-F1 score for the classification task in the ImageNet-pretrained setting. This discrepancy indicates that improved global embedding structure does not necessarily translate into better classification performance. One possible explanation is that the classifier head, optimization objective, and class imbalance influence decision boundaries more strongly than the overall geometry of the feature space.

4.4 Overall analysis of the results

Overall, the results indicate that unlabeled objectives can improve representation quality, but their effect on downstream classification depends on how effectively the classifier exploits these representations.

For each initialization setting (ImageNet-pretrained and random initialization), we now provide a more detailed analysis of the strongest-performing model. Figure 10 shows the confusion matrices for the supervised baseline in the ImageNet-pretrained setting and the SimCLR self-supervised method in the random-initialization setting. Table 4 provides the corresponding per-class precision, recall, and F1 scores.

The results in Table 4 and Figure 10 show that low-sample classes such as **OtherAB**, **UndefAB**, and **NonAB** are the most challenging categories in both settings. Among these classes, **OtherAB** achieves the lowest and most variable F1 scores (supervised with ImageNet: 0.469 ± 0.312 ; SimCLR with random initialization: 0.297 ± 0.194). This behavior is consistent with both its limited sample size and its heterogeneous class definition.

The results further suggest that when evaluation is restricted to the primary plaque classes (**Diffuse**, **Cored**, **Compact**, **Coarse**, **CAA**, and **Subpial**), each of which contains more than 10 annotated samples, the macro-F1 score increases substantially: from approximately 0.85 to 0.92 for the supervised baseline and from approximately 0.75 to 0.86 for SimCLR.

The confusion matrix also highlights systematic confusion between **Cored** and **Coarse**. This observation is consistent with their morphological similarity, as both represent dense plaque structures with subtle differences in compactness.

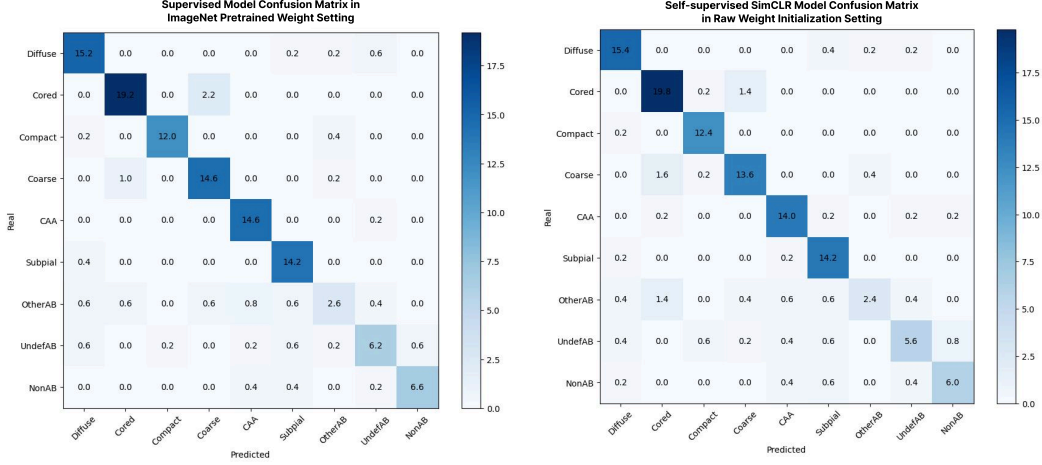


Figure 10: Average confusion matrix of the best-performing model across the 5-fold cross-validation test splits under the two initialization settings. Rows correspond to ground-truth classes and columns correspond to predicted classes. The matrix highlights systematic confusion between **Cored** and **Coarse**, which are morphologically similar plaque types, as well as instability for the heterogeneous **OtherAB** category.

Table 4: Per-class classification report for the strongest model under each initialization setting: the supervised baseline with ImageNet-pretrained ResNet-18 weights, and SimCLR (pretraining followed by supervised fine-tuning) under random initialization. Columns P, R, and F1 denote precision, recall, and F1-score. Values are reported as mean $\pm$ standard deviation across the five cross-validation test folds. The lowest per-class F1 in each model column (**OtherAB**) is highlighted in bold. Support refers to the labeled test instances aggregated across folds.

Class	Supervised (ImageNet)			SimCLR (random initialization)			Support
	P	R	F1	P	R	F1	
Diffuse	0.892 $\pm$ 0.032	0.938 $\pm$ 0.108	0.912 $\pm$ 0.067	0.857 $\pm$ 0.110	0.878 $\pm$ 0.071	0.862 $\pm$ 0.054	16.2 $\pm$ 0.447
Cored	0.927 $\pm$ 0.048	0.898 $\pm$ 0.074	0.910 $\pm$ 0.038	0.838 $\pm$ 0.037	0.823 $\pm$ 0.104	0.828 $\pm$ 0.061	21.4 $\pm$ 0.548
Compact	0.985 $\pm$ 0.034	0.953 $\pm$ 0.043	0.967 $\pm$ 0.018	0.906 $\pm$ 0.120	0.937 $\pm$ 0.101	0.916 $\pm$ 0.085	12.6 $\pm$ 0.548
Coarse	0.845 $\pm$ 0.074	0.924 $\pm$ 0.032	0.881 $\pm$ 0.043	0.836 $\pm$ 0.073	0.898 $\pm$ 0.105	0.860 $\pm$ 0.035	15.8 $\pm$ 0.447
CAA	0.916 $\pm$ 0.064	0.987 $\pm$ 0.030	0.949 $\pm$ 0.042	0.765 $\pm$ 0.068	0.852 $\pm$ 0.086	0.802 $\pm$ 0.047	14.8 $\pm$ 0.447
Subpial	0.893 $\pm$ 0.081	0.972 $\pm$ 0.038	0.929 $\pm$ 0.047	0.870 $\pm$ 0.050	0.929 $\pm$ 0.072	0.899 $\pm$ 0.057	14.6 $\pm$ 0.548
OtherAB	0.617 $\pm$ 0.371	0.419 $\pm$ 0.363	0.469 $\pm$ 0.312	0.467 $\pm$ 0.361	0.224 $\pm$ 0.142	0.297 $\pm$ 0.194	6.2 $\pm$ 0.447
UndefAB	0.818 $\pm$ 0.172	0.717 $\pm$ 0.115	0.763 $\pm$ 0.138	0.628 $\pm$ 0.104	0.533 $\pm$ 0.165	0.573 $\pm$ 0.135	8.6 $\pm$ 0.548
NonAB	0.921 $\pm$ 0.072	0.871 $\pm$ 0.089	0.891 $\pm$ 0.039	0.755 $\pm$ 0.053	0.711 $\pm$ 0.204	0.717 $\pm$ 0.127	7.6 $\pm$ 0.548
Macro avg	0.868 $\pm$ 0.040	0.853 $\pm$ 0.034	0.852 $\pm$ 0.033	0.769 $\pm$ 0.061	0.754 $\pm$ 0.037	0.750 $\pm$ 0.046	117.8 $\pm$ 0.447
Weighted avg	0.889 $\pm$ 0.025	0.893 $\pm$ 0.019	0.885 $\pm$ 0.022	0.804 $\pm$ 0.037	0.810 $\pm$ 0.027	0.798 $\pm$ 0.031	117.8 $\pm$ 0.447

4.5 Comparison to previous work

In the previous work by Moreira-Kanaley [14], the authors reported a macro-F1 score of approximately 77% for the plaque-type classification task under a supervised learning setting. Although the supervised baseline in the present work achieves a higher macro-F1 score (0.852 ± 0.033), this should not be interpreted as a direct like-for-like comparison because the experimental settings differ in several important ways.

- The present study uses a larger labeled dataset (589 annotated plaques compared with 315 in the previous work).
- Input images are resized to a higher spatial resolution (224×224 compared with 128×128).
- Training and evaluation protocols differ between the two studies.

- The backbone architectures and initialization strategies also differ (ResNet-18 with ImageNet pretraining in this work versus an ensemble of ResNet-50 models with MoCo pretraining in the earlier study).

The primary objective of the present work was therefore not to necessarily outperform prior results, but rather to investigate how unlabeled plaque images influence representation quality and downstream classification performance in a setting where labeled samples are limited but unlabeled data are abundant.

5 Conclusion

In this work, we investigated the impact of unlabeled plaque images on A β plaque classification. We compared a supervised ResNet-18 baseline against semi-supervised methods (Pi-Model, FixMatch, and Mean Teacher) and self-supervised methods (SimCLR and VAE) using the same labeled and unlabeled data pools. The self-supervised approaches used unlabeled data during a separate pretraining stage followed by supervised fine-tuning, whereas the semi-supervised approaches optimized supervised and unlabeled objectives jointly during training. All methods were evaluated under the same experimental protocol using Optuna-based hyperparameter tuning and 5-fold cross-validation.

The main finding of this work is that unlabeled objectives provide limited downstream macro-F1 improvement when the backbone is already initialized with strong ImageNet-pretrained weights. Under this setting, the supervised baseline achieved the highest macro-F1 score (0.852 ± 0.033), but the strongest unlabeled-data methods were closer than before (SimCLR 0.835 ± 0.054 , VAE 0.834 ± 0.047 , Mean Teacher 0.832 ± 0.017). A likely explanation is that ImageNet pretraining already provides strong low-level and mid-level visual representations, which reduces the amount of additional information that can be extracted from the unlabeled plaque images. In particular, Pi-Model and FixMatch remained clearly below the supervised baseline, while Mean Teacher stayed competitive but still lower on average. This suggests that jointly optimizing supervised and consistency objectives can interfere with the classification objective when the initialization is already strong.

At the same time, the feature-space analysis revealed a split trend in separability: supervised had the highest 2D-PCA silhouette (0.577), whereas VAE and SimCLR had the highest full-dimensional silhouettes (0.700 and 0.690), with Mean Teacher and FixMatch also above supervised in full-dimensional space. **This indicates that improved representation geometry does not necessarily translate into improved downstream classification performance.** One possible reason is that macro-F1 is strongly influenced by difficult and low-frequency plaque categories, where better global cluster separation alone may not be sufficient to improve class-wise decision boundaries.

The class-wise evaluation further showed that the models struggled most on minority plaque categories, particularly **OtherAB**, **UndefAB**, and **NonAB**, which had substantially fewer annotated samples compared to the major plaque classes. Across both initialization settings, these low-frequency classes achieved considerably lower F1 scores, in some cases ranging between 0.30 and 0.80, whereas the dominant plaque categories generally achieved F1 scores above 0.80 in the best-performing models. This behavior highlights the sensitivity of macro-F1 to class imbalance and suggests that limited annotation availability for rare plaque types remains a major bottleneck for reliable classification performance.

A different trend emerged when the backbone was trained without ImageNet-pretrained initialization. In this setting, unlabeled objectives became substantially more beneficial. The SimCLR-based self-supervised approach achieved the best performance (0.750 ± 0.046), outperforming the supervised-from-scratch baseline (0.714 ± 0.034) and VAE (0.735 ± 0.037), while Mean Teacher and FixMatch reached intermediate performance (0.725 ± 0.035 and 0.725 ± 0.061). **These results suggest that unlabeled data become significantly more valuable when strong pretrained representations are not available.** In such settings, self-supervised pretraining can help the model learn useful visual structure before fine-tuning on the limited labeled dataset.

Overall, the findings of this work suggest that the effectiveness of unlabeled learning objectives strongly depends on the initialization regime and the quality of the pretrained backbone. When strong pretrained weights are available, supervised fine-tuning remains the dominant factor for downstream performance. However, when training from scratch or in settings where pretrained models are unavailable or less suitable, unlabeled data can provide meaningful improvements in representation learning and classification performance.

Finally, although the evaluated unlabeled learning strategies did not outperform the supervised ImageNet-pretrained baseline in the current setup, this does not imply that unlabeled learning is ineffective for plaque classi-

fication in general. The benefit of unlabeled data may depend on factors such as model architecture, augmentation policies, dataset scale, unlabeled corpus diversity, optimization strategy, and the amount of labeled supervision available. These directions, together with additional limitations of the present work, are discussed further in Section 6.

6 Limitations and Future Work

This study provides insights into the role of unlabeled data in $A\beta$ plaque classification, but several limitations suggest directions for future work.

First, all experiments are based on a ResNet-18 backbone. While efficient and widely used, its limited capacity may restrict the capture of fine-grained morphological features in plaque images. This choice was also driven by computational constraints, as the full experimental pipeline with 5-fold cross-validation and Optuna tuning required substantial training time per method (2 hours per full). Future work should explore larger architectures such as ResNet-34, ResNet-50, or vision transformers, as well as pathology-specific foundation models like H-Optimus-1 [2].

Second, only a small subset of the available unlabeled data was used (10,000 out of approximately 4.7 million plaque crops). Since self-supervised methods typically benefit from large-scale data, future studies should investigate scaling effects and whether better representations emerge from using the full dataset. The quality of automatically segmented crops should also be examined, as segmentation noise may affect learning.

Third, augmentation strategies were not fully optimized. Although standard policies from methods such as SimCLR and FixMatch were used [22, 4], no automated or pathology-specific augmentation search was performed. Future work could explore learned augmentation policies and their interaction with different initialization regimes.

Fourth, the hyperparameter search was limited to 20 Optuna trials per method, which may not fully capture complex interactions between loss weights, confidence thresholds, and training dynamics, especially in semi-supervised settings. More extensive or multi-stage optimization could improve fairness and performance comparisons.

Finally, class imbalance remains a key challenge. Rare and heterogeneous classes such as **OtherAB**, **UndefAB**, and **NonAB** consistently show lower and more unstable performance, which strongly affects macro-F1. Future work should address this through additional annotations, improved class definitions, or hierarchical classification schemes.

7 Acknowledgments

I would like to thank the supervisor Xucong Zhang and the supervising professor Marcel Reinders for their guidance and feedback throughout this project.

The plaque corpus builds on amyloid- β segmentations and study metadata introduced in prior work by Moreira-Kanaley [14]. I acknowledge the shared preprocessing pipeline and derived datasets provided by that work. The underlying tissue samples and imaging data were made available through Amsterdam UMC and study participants. I am also grateful to neuropathology expert Susanne K. Rhode for providing the manual plaque-type annotations used in this study.

Computational resources for hyperparameter optimization and model training were provided through institutional infrastructure and external GPU resources, as detailed in Appendix A.1.

AI Assistance Disclosure

Coding and exploratory analysis were supported by Cursor, an AI-assisted development environment used for implementation support and code refactoring. The written report was revised with assistance from ChatGPT for grammar, clarity, and stylistic consistency. All experimental design choices, technical content, and final interpretations remain the responsibility of the author.

References

- [1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, pages 2623–2631, 2019.
- [2] Bioptimus. H-optimus-1: Pathology foundation model weights and documentation. <https://huggingface.co/bioptimus/H-optimus-1>, 2025.
- [3] Baayla DC Boon, Marjolein Bulk, Allert J Jonker, Tjado HJ Morrema, Emma van den Berg, Marko Popovic, Jochen Walter, Sathish Kumar, Sven J van der Lee, Henne Holstege, et al. The coarse-grained plaque: a divergent $\alpha\beta$ plaque-type in early-onset alzheimer’s disease. *Acta neuropathologica*, 140(6):811–830, 2020.
- [4] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *International conference on machine learning*, pages 1597–1607. PmLR, 2020.
- [5] Andrew Collette et al. Hdf5 for python (h5py). <https://www.h5py.org>, 2014.
- [6] Ekin D Cubuk, Barret Zoph, Dandelion Mane, Vijay Vasudevan, and Quoc V Le. Autoaugment: Learning augmentation policies from data. *arXiv preprint arXiv:1805.09501*, 2018.
- [7] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255. Ieee, 2009.
- [8] William Falcon and The PyTorch Lightning Team. Pytorch lightning. <https://www.pytorchlightning.ai>, 2019.
- [9] Charles R Harris, K Jarrod Millman, Stéfan J Van Der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J Smith, et al. Array programming with numpy. *nature*, 585(7825):357–362, 2020.
- [10] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [11] Diederik P Kingma and Max Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.
- [12] Samuli Laine and Timo Aila. Temporal ensembling for semi-supervised learning. *arXiv preprint arXiv:1610.02242*, 2016.
- [13] Wes McKinney et al. Data structures for statistical computing in python. *scipy*, 445(1):51–56, 2010.
- [14] Janaína Moreira-Kanaley. Quantification of amyloid-beta plaques using few-shot learning. Master’s thesis, Delft University of Technology, Delft, The Netherlands, 2024.
- [15] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.
- [16] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in python. *the Journal of machine Learning research*, 12:2825–2830, 2011.
- [17] PyTorch Contributors. Torchvision. <https://github.com/pytorch/vision>, 2024. PyTorch ecosystem package for datasets, models, and image transforms.
- [18] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *International Conference on Medical image computing and computer-assisted intervention*, pages 234–241. Springer, 2015.

- [19] Peter J Rousseeuw. Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. *Journal of computational and applied mathematics*, 20:53–65, 1987.
- [20] Alberto Serrano-Pozo, Matthew P Frosch, Eliezer Masliah, and Bradley T Hyman. Neuropathological alterations in alzheimer disease. *Cold Spring Harbor perspectives in medicine*, 1(1):a006189, 2011.
- [21] Connor Shorten and Taghi M Khoshgoftaar. A survey on image data augmentation for deep learning. *Journal of big data*, 6(1):1–48, 2019.
- [22] Kihyuk Sohn, David Berthelot, Nicholas Carlini, Zizhao Zhang, Han Zhang, Colin A Raffel, Ekin Dogus Cubuk, Alexey Kurakin, and Chun-Liang Li. Fixmatch: Simplifying semi-supervised learning with consistency and confidence. *Advances in neural information processing systems*, 33:596–608, 2020.
- [23] Antti Tarvainen and Harri Valpola. Mean teachers are better role models: Weight-averaged consistency targets improve semi-supervised deep learning results. *Advances in neural information processing systems*, 30, 2017.
- [24] Lary C Walker. A β plaques. *Free neuropathology*, 1:31, 2020.
- [25] Haruyasu Yamaguchi, Yoichi Nakazato, Tsuneo Yamazaki, Mikio Shoji, Takeshi Kwarabayashi, and Shunsaku Hirai. Subpial β /a4 amyloid deposition occurs between astroglial processes in alzheimer-type dementia. *Neuroscience letters*, 123(2):217–220, 1991.

A Appendix

A.1 Experimental Setup Details

Software and Libraries All source code for this work is written in Python 3.10. The core libraries include `torch` (PyTorch [15]) and `pytorch-lightning` [8] for model development and training, `torchvision` [17] for pre-processing and augmentation, `scikit-learn` [16] for evaluation, `numpy` [9] and `pandas` [13] for data processing, `h5py` [5] for HDF5 data access, and `optuna` for hyperparameter optimization [1].

Hardware Most experiments were run on the Delft HPC cluster (Quadro K2200 GPU with 4 GB VRAM and 16 GB system memory). Long hyperparameter tuning runs were executed on a cloud virtual machine (NVIDIA GeForce RTX 4060 with 8 GB VRAM and 16 GB system memory) via Vast.ai. The data used in the cloud experiments was anonymized and the virtual machine instance was destroyed after the experiments were completed to clean up any trace of the data. On this hardware, a single 5-fold cross-validation run for the longest running methods which was the self-supervised methods took approximately 2 hours.

Reproducibility Random seeds were fixed for all reported runs (`random_seed=44`), including sampling, augmentation, and initialization steps where applicable. To support auditability, each run directory stores a copy of the resolved configuration used for that run. An example of the configuration file format is provided in Figure 11.

Moreover for each run the classification performance report and the confusion matrix are saved in a run directory for future reference.

A.2 Data Augmentation

Data augmentation is a technique used to increase the diversity of the training data by increasing the number of training samples by applying random transformations to the images. It is a crucial step in the training of deep learning models as it helps to prevent overfitting and improve the generalization performance of the model. In this work, we apply the following data augmentation techniques to the labeled and unlabeled plaque images:

For the labeled data, we apply a simple augmentation pipeline which includes simple geometric transformations (e.g. flip and rotate) and photometric transformations (e.g. color jitter). This choice is consistent with prior work showing that combining geometric and photometric perturbations improves generalization in visual recognition tasks [21, 6].

```

{
  "model_name": "pi_model",
  "training":{
    "consistency_lambda_max": 0.001,
    "consistency_loss_type": "mse",
    "ramp_up_epochs": 100,
    "ramp_up_function": "linear"
  },
  "hyperparameter_tuning":{
    "training":{
      "consistency_lambda_max": [0.001, 0.005, 0.01],
      "ramp_up_epochs": [50, 100],
      "ramp_up_function": ["linear", "sigmoid", "fixed"],
      "consistency_loss_type": ["mse", "kl"]
    }
  }
}

```

Figure 11: Example JSON configuration file format for the semi-supervised Pi-Model. The configuration includes both standard training parameters and explicit search spaces for hyperparameter tuning (see Section 3.1).

- Random horizontal flip with a probability of 0.5.
- Random vertical flip with a probability of 0.5.
- Random rotation between -90 and 90 degrees.
- Color jitter with brightness and contrast values of 0.2.

For the unlabeled data, for each learning paradigm, we apply a different augmentation pipeline:

1. Semi-supervised learning: we draw inspiration from FixMatch [22], using a weak view for pseudo-labeling and a strong view for consistency.
 - Weak view augmentations (for pseudo-labeling):
 - Random horizontal flip with a probability of 0.5.
 - Random vertical flip with a probability of 0.5.
 - Random resized crop to the target image size, with scale sampled uniformly from [0.8, 1.0].
 - Strong view augmentations (for consistency regularization):
 - Random horizontal flip with a probability of 0.5.
 - Random vertical flip with a probability of 0.5.
 - Random resized crop to the target image size, with scale sampled uniformly from [0.8, 1.0].
 - RandAugment operation (with 2 transforms and magnitude 10).
2. Self-supervised learning: we follow a SimCLR-style two-view pipeline [4] with two independent strong augmentations per image.
 - Random resized crop to the target image size, with scale sampled uniformly from [0.6, 1.0].
 - Random horizontal flip with a probability of 0.5.
 - Random vertical flip with a probability of 0.5.
 - Random apply of color jitter (brightness 0.4, contrast 0.4, saturation 0.4, hue 0.1) with probability 0.8.
 - Random grayscaling with a probability of 0.2.
 - Gaussian blur with kernel size 3 and sigma sampled from [0.1, 2.0].

A.3 Hyperparameter Search

All experiments use the AdamW optimizer. Training is capped at 200 epochs, but early stopping may terminate training earlier if the validation macro-F1 score does not improve. Validation is performed every five epochs (rather than after each epoch), and early stopping decisions are based on this evaluation schedule.

Hyperparameter tuning is performed using Optuna with a Tree-structured Parzen Estimator (TPE) sampler, with 20 trials per method. The tables below report (i) the defined search space and (ii) the best hyperparameter configuration selected for each method.

Supervised (ResNet-18)

Section	Hyperparameter	Values	Selected	Description
training	learning_rate	[1e-5, 5e-5]	1e-5	AdamW learning rate.
training	batch_size	[32, 64]	32	Labeled mini-batch size.
training	weight_decay	[1e-5, 1e-4]	1e-4	AdamW weight decay.
feature_extractor	freeze	[true, false]	false	Freeze backbone or not.
feature_extractor	output_size	[32, 64, 128]	64	Projected feature dimension.
feature_extractor	dropout_rate	[0.1, 0.2, 0.3]	0.1	Dropout before classifier.

Pi-Model (Semi-supervised, ResNet-18)

Section	Hyperparameter	Values	Selected	Description
training	learning_rate	[1e-5, 5e-5]	1e-5	AdamW learning rate.
training	batch_size	[32, 64]	32	Labeled mini-batch size.
training	weight_decay	[1e-5, 1e-4]	1e-5	AdamW weight decay.
feature_extractor	freeze	[true, false]	false	Freeze backbone or not.
feature_extractor	output_size	[32, 64, 128]	64	Projected feature dimension.
feature_extractor	dropout_rate	[0.1, 0.2, 0.3]	0.3	Dropout before classifier.
semi_supervised	consistency_lambda_max	[0.001, 0.005, 0.01]	0.01	Max consistency weight.
semi_supervised	ramp_up_epochs	[50, 100]	100	Ramp-up duration.
semi_supervised	ramp_up_function	[linear, sigmoid, fixed]	sigmoid	Ramp-up schedule type.
semi_supervised	consistency_loss_type	[mse, kl, ce]	mse	Consistency criterion. (MSE: Mean Squared Error, KL: Kullback-Leibler Divergence, Cross-Entropy: Cross Entropy Loss)
pi_model	-	-	-	No additional Pi-Model-only tuned parameter.

FixMatch (Semi-supervised, ResNet-18)

Section	Hyperparameter	Values	Selected	Description
training	learning_rate	[1e-5, 5e-5]	1e-4	AdamW learning rate.
training	batch_size	[32, 64]	64	Labeled mini-batch size.
training	weight_decay	[1e-5, 1e-4]	1e-4	AdamW weight decay.
feature_extractor	freeze	[true, false]	false	Freeze backbone or not.
feature_extractor	output_size	[32, 64, 128]	64	Projected feature dimension.
feature_extractor	dropout_rate	[0.1, 0.2, 0.3]	0.2	Dropout before classifier.
semi_supervised	consistency_lambda_max	[0.001, 0.005, 0.01]	0.01	Max consistency weight.
semi_supervised	ramp_up_epochs	[50, 100]	50	Ramp-up duration.
semi_supervised	ramp_up_function	[linear, sigmoid, fixed]	sigmoid	Ramp-up schedule type.
semi_supervised	consistency_loss_type	[kl, ce]	kl	Consistency criterion. (KL: Kullback-Leibler Divergence, Cross-Entropy: Cross Entropy Loss)
fixmatch	pseudo_label_confidence_threshold	[0.85, 0.9, 0.95]	0.95	Minimum confidence to keep pseudo-labels.

Mean Teacher (Semi-supervised, ResNet-18)

Section	Hyperparameter	Values	Selected	Description
training	learning_rate	[1e-5, 5e-5]	5e-5	AdamW learning rate.
training	batch_size	[32, 64]	64	Labeled mini-batch size.
training	weight_decay	[1e-5, 1e-4]	1e-4	AdamW weight decay.
feature_extractor	freeze	[true, false]	false	Freeze backbone or not.
feature_extractor	output_size	[32, 64, 128]	128	Projected feature dimension.
feature_extractor	dropout_rate	[0.1, 0.2, 0.3]	0.1	Dropout before classifier.
semi_supervised	consistency_lambda_max	[0.001, 0.005, 0.01]	0.005	Max consistency weight.
semi_supervised	ramp_up_epochs	[50, 100]	100	Ramp-up duration.
semi_supervised	ramp_up_function	[linear, sigmoid, fixed]	sigmoid	Ramp-up schedule type.
semi_supervised	consistency_loss_type	[mse, kl, ce]	kl	Consistency criterion. (MSE: Mean Squared Error, KL: Kullback-Leibler Divergence, Cross-Entropy: Cross Entropy Loss)
mean_teacher	ema_decay	[0.95, 0.99]	0.95	EMA smoothing for teacher updates.

SimCLR (Self-supervised, ResNet-18)

Section	Hyperparameter	Values	Selected	Description
training	learning_rate	[1e-5, 5e-5]	1e-5	Downstream AdamW learning rate.
training	batch_size	[32, 64]	32	Downstream labeled mini-batch size.
training	weight_decay	[1e-5, 1e-4]	1e-5	Downstream AdamW weight decay.
feature_extractor	freeze	[true, false]	false	Downstream backbone freezing.
feature_extractor	output_size	[32, 64, 128]	32	Projected feature dimension.
feature_extractor	dropout_rate	[0.1, 0.2, 0.3]	0.1	Dropout before classifier.
self_supervised	pretraining.num_epochs	[50, 100]	50	Number of pretraining epochs.
self_supervised	pretraining.learning_rate	[1e-5, 5e-5, 1e-4]	1e-5	Pretraining learning rate.
self_supervised	pretraining.weight_decay	[1e-5, 1e-4]	1e-4	Pretraining weight decay.
self_supervised	pretraining.freeze	[true, false]	false	Pretraining freezing flag.
simclr	temperature	[0.05, 0.1, 0.2]	0.2	Contrastive temperature.
simclr	projection_head_sizes	[[128], [512, 128]]	[128]	One-layer vs. two-layer head architectures.
simclr	projection_head_activation	[relu, tanh]	relu	Projection head nonlinearity.

VAE (Self-supervised, ResNet-18)

Section	Hyperparameter	Values	Selected	Description
training	learning_rate	[1e-5, 5e-5]	1e-5	Downstream AdamW learning rate.
training	batch_size	[32, 64]	32	Downstream labeled mini-batch size.
training	weight_decay	[1e-5, 1e-4]	1e-4	Downstream AdamW weight decay.
feature_extractor	freeze	[true, false]	false	Downstream backbone freezing.
feature_extractor	output_size	[32, 64, 128]	32	Projected feature dimension.
feature_extractor	dropout_rate	[0.1, 0.2, 0.3]	0.1	Dropout before classifier.
self_supervised	pretraining.num_epochs	[50, 100]	50	Number of pretraining epochs.
self_supervised	pretraining.learning_rate	[1e-5, 5e-5, 1e-4]	1e-5	Pretraining learning rate.
self_supervised	pretraining.weight_decay	[1e-5, 1e-4]	1e-4	Pretraining weight decay.
self_supervised	pretraining.freeze	[true, false]	false	Pretraining freezing flag.
vae	latent_dim	[32, 64]	32	Latent representation dimensionality.
vae	beta	[0.5, 0.75, 1.0]	1.0	Weight of KL divergence term.
vae	reconstruction_loss	mse	mse	Pixel-wise reconstruction criterion.

A.4 Code Availability

The source code, configuration files, and experimental results are available at <https://github.com/ParhamBateni/amyloid-beta-plaque-classification>.